DOCUMENTO DE REQUISITOS E MELHORIAS — AVALIAEDU

Especificação de Escopo, Transformação Arquitetural (AS-IS → TO-BE) e Registro de Correções Técnicas

Integrante / Equipe	Papel e Escopo de Atuação no Projeto
Bruna	Desenvolvimento Back-end (Laravel), integração com chatbot, Code Reviews e **Auditoria Arquitetural / Refatoração Técnica (AS-IS → TO-BE em 01/11/2026)**.
Beatriz	Levantamento e documentação inicial de requisitos (v1.0), telas Front-end e modelagem do sistema.
Laiane	Design das telas no Figma (v1.0), definição do fluxo do chatbot via WhatsApp e testes da aplicação.
Luiz	Desenvolvimento Front-end (v1.0), modelagem do banco de dados e organização do Kanban.

Registro de Alteração	Descrição da Consolidação e Melhorias	Data
Refatoração & Melhorias MVP	Auditoria arquitetural completa do AvaliaEDU, mapeamento de viabilidade (AS-IS para TO-BE), consolidação dos 5 perfis institucionais e aplicação das 16 correções críticas de Back-end, Banco de Dados (SQLite/MySQL), Autenticação Fortify e componentes Livewire 3 / PowerGrid.	01/11/2026

Sumário Executivo

1. Contexto de Viabilização do Projeto e Mudanças (AS-IS → TO-BE) **2**
2. Escopo Consolidado e Perfis de Acesso do AvaliaEDU **3**
3. Módulos do Sistema e Diagrama de Casos de Uso **4**
4. Requisitos Funcionais, Não Funcionais e Regras de Negócio **5**
5. Registro Detalhado dos 16 Itens Corrigidos na Refatoração (01/11/2026) **6**

Resumo do Objetivo

O **AvaliaEDU** é um sistema web institucional de avaliações educacionais semestrais e ouvidoria escolar desenvolvido com Laravel 12, Livewire 3, PowerGrid e Tailwind CSS. Este documento registra as decisões de engenharia adotadas em 01/11/2026 para eliminar dívidas técnicas da versão inicial (v1.0), tornar a arquitetura sustentável e garantir execução local e em produção sem falhas.

1. Contexto de Viabilização Técnica (AS-IS → TO-BE)

Durante a revisão arquitetural da base original do TCC (marcada na tag **v1.0**), identificou-se que algumas implementações iniciais impediam o funcionamento ponta a ponta ou geravam gargalos severos de performance e segurança. A tabela abaixo sintetiza a evolução do estado original (**AS-IS**) para a arquitetura estabilizada (**TO-BE**) entregue nesta versão:

Dimensão	Como Era (AS-IS — Legado v1.0)	Como Ficou (TO-BE — Melhorias 01/11/2026)
1. Fluxo de Primeiro Acesso (Fortify)	A classe *CreateNewUser* referenciava um Model inexistente (*PreRegistro*), validava a tabela no singular (*pre_registro*) e usava a variável *$preRegistro* sem buscá-la no banco. No front-end, o regex bloqueava e-mails de alunos, direção e coordenação.	Unificação no Model **PreCadastro** (tabela *pre_registros*), busca com bloqueio pessimista (*lockForUpdate*) dentro de *DB::transaction* em *CreateNewUser* e liberação dos 5 domínios institucionais no regex de validação.
2. Geração de Matrículas e Concorrência	O método *Curso::gerarMatriculaUnica()* executava *$curso->save()* repetidamente dentro de um loop *do...while* sem transação, vulnerável a condições de corrida (*Race Condition*).	Encapsulamento em **DB::transaction()** com **lockForUpdate()** na linha do curso e um único *$curso->save()* ao final do cálculo, garantindo atomicidade ACID.
3. Performance SQL e Queries N+1	Importação de planilhas executava *Role::where()* a cada linha; cards de ouvidoria disparavam 3 queries *COUNT(*)* separadas; gráficos de avaliação carregavam todas as respostas na memória RAM.	Pré-carregamento com *Role::pluck('id', 'acesso')* fora do loop; consolidação dos 3 contadores em 1 única query SQL com *SUM(CASE WHEN...)*; agregação via *GROUP BY* direto no banco.
4. Isolamento Multi-Tenant por Escola	A importação em lote fixava *escola_id = 1* hardcoded, e as tabelas PowerGrid listavam cursos, alunos e pré-cadastros de todas as escolas misturados.	Vínculo dinâmico com *Auth::user()->escola_id* e aplicação do **scopeDaEscola()** em todas as listagens (*CursoTable, AlunoTable, PreCadastroTable, PreRegistroTable*).
5. Padrão PSR-4 e Livewire 3	Colisão de classe (*DashboardProfessor.php* declarava *class CursoEditModal*), uso de namespaces antigos do Livewire 2 (*App\Http\Livewire*), *emitSelf* removido e *sleep(3)* travando a thread do servidor PHP.	Alinhamento 100% ao autoloader PSR-4 (*namespace App\Livewire*), migração para eventos *#[On]* e *$this->dispatch()* do Livewire 3 e remoção do *sleep(3)* bloqueante.
6. Portabilidade de Banco (SQLite / MySQL)	O seeder de avaliações usava comando exclusivo do MySQL (*SET FOREIGN_KEY_CHECKS=0*), impedindo a execução rápida em SQLite local.	Uso de *Schema::disableForeignKeyConstraints()* agnóstico de banco, permitindo rodar todo o sistema localmente com zero configuração externa via SQLite.

2. Escopo Consolidado e Perfis de Acesso do AvaliaEDU

O **AvaliaEDU** estrutura suas permissões e redirecionamentos pós-login (via *LoginResponse*) em **5 perfis institucionais principais** (além da conta de suporte técnico/admin):

1. Secretaria (@secretaria.gov.br)

- Gerenciar os cursos ativos da unidade escolar (criação, edição e listagem via PowerGrid);
- Realizar o pré-cadastro individual ou em lote (importação de planilha Excel/CSV) de alunos e servidores;
- Gerar matrículas únicas sequenciais por curso e acompanhar o status de ativação das contas.

2. Direção (@direcao.gov.br)

- Visualizar o painel estratégico da escola com indicadores de manifestações (Total, Pendentes e Resolvidas);
- Gerenciar a abertura, vigência e encerramento dos ciclos semestrais de avaliação institucional;
- Acompanhar e responder manifestações registradas na ouvidoria da unidade escolar.

3. Coordenação (@coordenacao.gov.br)

- Acessar o painel de coordenação (rota corrigida para *perfis.coordenador.coordenador-home*);
- Acompanhar indicadores pedagógicos, desempenho dos cursos e demandas encaminhadas à coordenação.

4. Professor (@professor.gov.br)

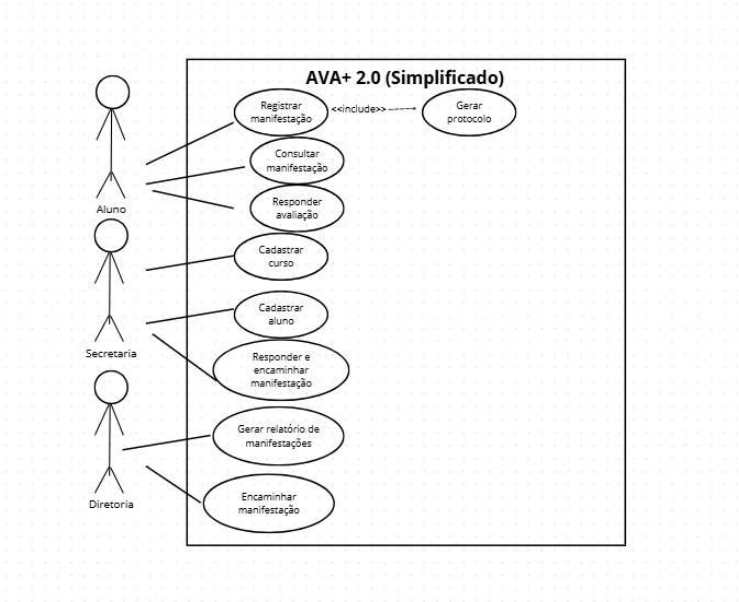
- Acessar o painel docente (*professor.painel*) com visualização consolidada dos gráficos de avaliação;
- Restrição explícita de segurança (HTTP 403 em *AvaliacaoController*) impedindo alteração indevida de ciclos de avaliação.

5. Aluno (@aluno.gov.br)

- Ativar sua conta no Primeiro Acesso com o e-mail institucional gerado no pré-cadastro;
- Participar das avaliações semestrais (mediante validação de matrícula ativa via *possuiVinculoAtivo()*) e registrar manifestações na ouvidoria.

3. Módulos do Sistema e Casos de Uso

Figura 1 – Diagrama de Casos de Uso (AvaliaEDU)



Fonte: Elaboração própria, AvaliaEDU.

Módulo de Gestão Acadêmica e Pré-Cadastro

- Cadastro de cursos e geração transacional de códigos de matrícula únicos por sigla de curso;
- Pré-cadastro individual e importação em lote via planilha (Maatwebsite Excel) com geração automática de e-mail institucional;
- Ativação de conta no Primeiro Acesso integrada ao Laravel Fortify.

Módulo de Avaliação Institucional Semestral

- Controle de abertura, vigência e fechamento de períodos avaliativos (ex.: 2025/1, 2025/2);
- Coleta de respostas anônimas estruturadas por perguntas de escala e múltipla escolha;
- Consolidação de indicadores em gráficos (participação, dificuldades, conforto para dúvidas e coerência avaliativa).

Módulo de Ouvidoria Escolar e Dashboards

- Registro e acompanhamento de reclamações e elogios vinculados à unidade escolar;
- Painéis reativos em Livewire 3 com contadores agregados em tempo real por escola.

4. Requisitos e Regras de Negócio Consolidados

Código	Requisito Funcional (RF)	Prioridade
RF01	Permitir que a Secretaria gerencie os cursos ativos vinculados à sua própria unidade escolar.	Alta
RF02	Permitir o pré-cadastro individual e via planilha (Excel/CSV) de alunos e servidores com geração automática de e-mail institucional.	Alta
RF03	Permitir a ativação da conta no Primeiro Acesso validando se o e-mail institucional consta como 'pendente' na tabela pre_registros.	Alta
RF04	Redirecionar automaticamente o usuário autenticado para o painel correspondente ao seu perfil (Secretaria, Direção, Coordenação, Professor ou Aluno).	Alta
RF05	Permitir que a gestão escolar abra, edite, conclua e encerre automaticamente por vencimento os ciclos semestrais de avaliação.	Alta
RF06	Validar se o aluno possui matrícula ativa (status 'ativa') antes de permitir operações que exijam vínculo escolar vigente.	Alta
RF07	Impedir a exclusão de alunos que possuam matrículas ativas vinculadas.	Alta
RF08	Apresentar dashboards de reclamações por escola (Total, Pendentes e Resolvidas) e gráficos agregados das respostas de avaliação.	Alta

Código	Requisito Não Funcional (RNF) / Regra de Arquitetura	Prioridade
RNF01	Isolamento Multi-Tenant: Consultas de listagem e exclusão devem respeitar o escola_id do usuário logado.	Alta
RNF02	Performance SQL: Proibido o uso de queries dentro de loops (N+1) ou carregamento integral de tabelas de respostas na memória RAM.	Alta
RNF03	Segurança e Rate Limiting: O endpoint público de verificação de e-mail (/register/check-email) deve possuir proteção throttle (10 req/min).	Alta
RNF04	Concorrência ACID: Operações críticas de ativação de conta e geração de matrícula devem utilizar DB::transaction e lockForUpdate.	Alta
RNF05	Padrão PSR-4 e Tipagem: Todas as classes devem respeitar o autoloader PSR-4 e utilizar tipagem explícita do PHP 8.2+.	Média

5. Registro dos Itens Corrigidos na Refatoração (01/11/2026)

Abaixo estão listados todos os **16 itens técnicos** corrigidos no código-fonte do **AvaliaEDU** em 01/11/2026 para estabilização completa do Back-end e funcionamento das Views:

#	Arquivo(s) Impactado(s)	Correção Aplicada em 01/11/2026
01	**app/Models/Matricula.php**	Inclusão de curso_id no $fillable, adição dos relacionamentos turma() e reclamacoes(), tipagem estrita (BelongsTo, HasMany, Builder) e correção dos Scopes para os valores reais do banco ('ativa', 'trancada', 'concluida', 'transferido').
02	**app/Models/Aluno.php**	Adição de cast 'date' para alun_data_nascimento, uso de latestOfMany('data_matricula') em matriculaAtiva(), correção do filtro 'ativo' para 'ativa' em cursoAtual() e criação do método possuiVinculoAtivo(): bool.
03	**app/Models/Curso.php**	Refatoração de gerarMatriculaUnica() com DB::transaction() e lockForUpdate() (eliminando save() dentro do loop do...while), adição do relacionamento turmas() e criação do scopeDaEscola().
04	**app/Models/PreCadastro.php**	Padronização da constante DOMINIOS_POR_PERFIL para os 5 perfis, uso do operador nullsafe (?->) em Auth::user()?->name, adição de scopeAtivados() e correção da FK 'pre_registro_id' em hasOne(User::class).
05	**app/Models/User.php**	Correção do relacionamento preRegistro()/preCadastro() apontando para PreCadastro::class com FK 'pre_registro_id', inclusão de pre_registro_id no $fillable e criação do helper variádico possuiPerfil(string ...$perfis).
06	**app/Models/PerguntasAvaliacao.php & RespostasAvaliacao.php**	Correção dos nomes das classes nos relacionamentos (que estavam no singular e causavam Class Not Found) e substituição do carregamento em memória por agregação SQL GROUP BY em getDadosDashboard().
07	**app/Actions/Fortify/CreateNewUser.php**	Correção de bug fatal onde $preRegistro não era consultado no banco antes de User::create, correção do nome da tabela na validação Rule::exists('pre_registros') e execução atômica via DB::transaction.
08	**app/Http/Responses/LoginResponse.php & FortifyServiceProvider.php**	Correção de erro fatal de sintaxe em LoginResponse.php (código solto fora do método toResponse), correção da rota admin.dashboard, inclusão do perfil aluno e registro via singleton no ServiceProvider.
09	**app/Imports/UsersImport.php & ImportarPreCadastro.php**	Substituição de PreRegistro e AtivacaoContaMail inexistentes por PreCadastro::criarComEmailInstitucional(), pré-carregamento de Role::pluck() fora do loop (fim do N+1) e remoção do escola_id = 1 fixo.
10	**resources/views/livewire/auth/register.blade.php**	Atualização do regex de validação de e-mail para aceitar @aluno.gov.br, @coordenacao.gov.br e @direcao.gov.br, e correção do atributo Alpine x-bind:class no componente <flux:input>.
11	**routes/web.php & SecretariaController.php**	Remoção do import duplicado de Role em SecretariaController, correção do typo da view 'perfis.coordenador.coordenador-home' no login da coordenação e adição de middleware throttle:10,1 em /register/check-email.
12	**app/Http/Controllers/AvaliacaoController.php & AvaliacoesStatus.php**	Correção da checagem de autorização (usando possuiPerfil('professor', 'aluno') em vez de comparar objeto Role com string), ajuste do namespace para App\Livewire e correção do nome da view 'livewire.avaliacoes-status'.

#	Arquivo(s) Impactado(s)	Correção Aplicada em 01/11/2026
13	**app/Livewire/DashboardProfessor.php & Toast.php**	Resolução da colisão fatal de classe PSR-4 em DashboardProfessor.php (que declarava CursoEditModal), migração de Toast.php para Livewire 3 ($this->dispatch) e remoção do sleep(3) bloqueante.
14	**app/Livewire/PreCadastroTable.php & PreRegistroTable.php**	Substituição das referências a PreRegistro e Roles inexistentes, adição de Eager Loading with(['escola', 'role']), filtro multi-tenant daEscola() e agrupamento seguro dos filtros orWhere.
15	**app/Livewire/AlunoTable.php, CursoTable.php & sec-home.blade.php**	Aplicação de filtro por escola_id nas tabelas de alunos e cursos, uso de possuiVinculoAtivo() antes da exclusão de aluno e correção do typo <livewire:curso-edit-modal /> em sec-home.blade.php.
16	**app/Livewire/ReclamacaodaMinhaEscola.php, ReclamacoesPorEscola.php & AvaliacaoSeeder.php**	Unificação de 3 queries COUNT(*) repetidas em 1 única query SQL agregada com SUM(CASE WHEN...), ajuste de capitalização PSR-4 e uso de Schema::disableForeignKeyConstraints() compatível com SQLite.